Sam

For my Heart Disease Prediction & Signal Analysis midterm project, I used Google Gemini in my Kaggle workflow to help troubleshoot syntax errors, structure advanced ensemble code, and figure out why my models were stuck around a 0.48 to 0.51 ROC-AUC score even after heavy tuning. What I really learned from this project is that no amount of algorithm complexity—whether it is applying SMOTE, adjusting class weights in XGBoost, running GridSearchCV, or building a soft-voting ensemble with XGBoost, LightGBM, and Random Forest—can pull a prediction out of data if the features have no real statistical connection to the target. By digging into the Pearson correlations and running SHAP TreeExplainer plots, I proved that all 20 patient features had near-zero linear correlation (within ±0.02) with the raw Heart Disease Status column, and that Stress Level had almost no impact (0.016) compared to clinical biomarkers like BMI, Homocysteine Level, and CRP Level. I also learned how important it is to split raw encoded data before fitting StandardScaler so test data never leaks into the training phase. Instead of just settling for a 0.50 random-guess model when my baseline Random Forest failed to catch the minority class, I took control of the notebook and built a 16-step investigation to test every angle. To make this project my own, I engineered custom cardiovascular risk features—including a Metabolic Risk Score, a Cardio Strain Index, an Inflammatory Vascular Score, and a consolidated Lipid Risk Score—while fixing duplicate column issues from degree-2 polynomial interactions and cleaning up feature names for LightGBM. Finally, I designed a custom signal verification experiment from scratch using realistic clinical weights for Age, BMI, Blood Pressure, Cholesterol, Homocysteine, Smoking, and Diabetes, paired with stratified shuffling, which proved my pipeline worked as intended when it jumped from ~0.50 on the noisy synthetic data to 0.9948 with my Soft-Voting Ensemble and 1.0000 ROC-AUC (100% accuracy, precision, and recall) with my final high-capacity Logistic Regression model

Ebuka

For this project, I worked with a Heart Attack Prediction dataset from Kaggle to practice cleaning and preparing data for machine learning. One of the biggest things I learned was how important preprocessing is before training a model. The dataset had 8,763 rows, which gave me a good amount of data to work with. I used Python in Jupyter Notebook to split the dataset into 70% training data and 30% testing data. I also learned why it's important to keep the testing data separate so the model can be evaluated fairly later on. I worked through different preprocessing techniques, including scaling numerical values, encoding categorical variables, checking for missing values, and balancing the target classes. I checked for missing values and learned how to handle them appropriately when needed. I also used Matplotlib to visualize the data before and after preprocessing, which made it easier to understand how the changes affected the dataset. This project helped me better understand how the concepts we've covered in class apply to an actual dataset. I also realized that not every preprocessing technique needs to be applied just because it's available. It depends on the dataset and what you're trying to accomplish. Overall, I feel more comfortable using Python to prepare data, and I have a better understanding of how these steps will help when I start training and comparing machine learning models.

Cersidy

The mid-term was insightful from data ingestion to creating a model. I learned different techniques like using median and most frequent in data preparation with simple imputer and one-hot encoding using scikit-learn to transform categorical data to numerical values the model could work with. Using a standard scaler to normalize numerical values to have a mean of 0 and a variance of 1. I used a pipeline technique to combine the preprocess and the algorithm. The end product was a model built on the heart disease csv file with 1000rows from patient records without exposing or violating HIPPA rules.

Sadia Ravate

Introduction

This midterm project focuses on preparing a heart disease dataset for machine learning. The purpose is to understand how data preprocessing and exploratory data analysis help prepare data for a prediction task. Working on this project has helped me connect the concepts learned in class with a practical application of artificial intelligence and machine learning.

What I Learned

One of the first things I learned was the importance of understanding a dataset before using it to build a machine learning model. Our dataset, `heart_disease.csv`, contains 10,000 records and 21 columns. Loading the dataset into Google Colab and examining its structure helped me understand how Python and pandas can be used to inspect data, identify columns, and determine which variables may be useful for analysis.

I also learned that real-world datasets may contain missing values. These missing values need to be identified and handled appropriately because incomplete data can affect the quality of an analysis. Depending on the type of variable, methods such as filling missing numerical values with the median or categorical values with the mode can be considered. This helped me understand why data cleaning is an important step before training a model.

Another important concept I learned was data preprocessing. Machine learning algorithms often require data to be represented in a suitable numerical format. Encoding can convert categorical variables into numerical values, while scaling can help put numerical features on comparable scales. I learned that these techniques must be selected based on the characteristics of the dataset and the requirements of the model.

The project also helped me understand exploratory data analysis (EDA). Examining distributions, reviewing sample records, and creating visualizations can reveal patterns, unusual values, and differences between variables. Visualizations are useful because they make information easier to interpret than looking at raw data alone.

Challenges and Problem-Solving

One challenge was working with the dataset in Google Colab and ensuring that the file was loaded from the correct location. I learned that file paths must match where the dataset is stored in the working environment. I also learned the importance of reading the output of commands such as `head()` and `info()` to confirm that the data has loaded correctly and to identify potential issues before continuing.

Another important lesson was understanding how to divide data into training and testing sets. The training set is used to develop the model, while the testing set is used to evaluate how well it performs on data it has not seen during training. For this project, the required split is 70% training data and 30% testing data. I learned that information from the test set should not influence preprocessing decisions or model development, because this could lead to data leakage and an unreliable evaluation.

Connection to Artificial Intelligence

This project demonstrates that building a machine learning model involves more than selecting an algorithm and running code. The quality of the input data, the preprocessing methods, and the evaluation process all affect how trustworthy the results are. In a healthcare-related prediction task, it is especially important to interpret model results carefully and avoid treating predictions as definitive medical diagnoses.

I also learned that high model accuracy alone does not necessarily prove that a model is reliable. The evaluation must use an appropriate target variable and a separate test set. The goal is to build an analysis that is methodologically sound rather than simply obtain a high score.

Conclusion

Overall, this midterm project has helped me develop a better understanding of data preparation for machine learning. I gained practical experience working with a dataset in Google Colab and learned why data inspection, missing-value handling, encoding, scaling, visualization, and proper training and testing procedures matter. I also gained a better appreciation for documenting each step so that the analysis can be understood and reproduced.

As I continue developing the notebook, I will focus on completing the preprocessing steps, verifying the required 70/30 split, comparing the data before and after cleaning, and ensuring that the final results are evaluated correctly. These skills will be useful in future AI and machine learning projects.